You can put any key or value into a map, but only atom keys can use the key: value shorthand. If you want to set a key that isn't an atom, then you have to use the => syntax, like this:

```
iex> map = %{1 => "an integer", 2 => %{another: :map}}

iex> Map.keys(map)
[1, 2]

iex> Map.values(map)
["an integer", %{another: :map}]

iex> Map.get(map, 1)
"an integer"

iex> map[1]
"an integer"

iex> map.1
# SyntaxError
```

Elixir's Map[3] module is used to read and modify maps. This is a commonly used module, so you should look over it to see what's available to you. There are lots of functions in the Map module, but here are a few of the basic ones:

```
iex> map = %{}
iex> map = Map.put(map, :key, :value)
%{key: :value}

iex> map2 = Map.put(map, "list", [])
%{key: :value, "list" => []}

iex> map
%{key: :value}

iex> [Map.get(map, :key), Map.get(map, :not_in_map)]
[:value, nil]

iex> Map.delete(map2, "list")
%{key: :value}
```

Maps don't have the same performance problem that lists have. They are implemented to be very efficient, so you can access or modify maps by key without concern.

The previous example highlights a core concept in Elixir: data is immutable. Map.put/3 was used on the map variable, but the original map wasn't changed.

Let's take a closer look at immutability.

3. https://hexdocs.pm/elixir/1.14/Map.html